

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»**

**Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем**

Лабораторна робота № 1

з навчальної дисципліни
«Технології на платформі .NET»

Виконала:

студентка 403-ТН

Кіріяк Ірина Вікторівна

Перевірив:

Тютюнникт

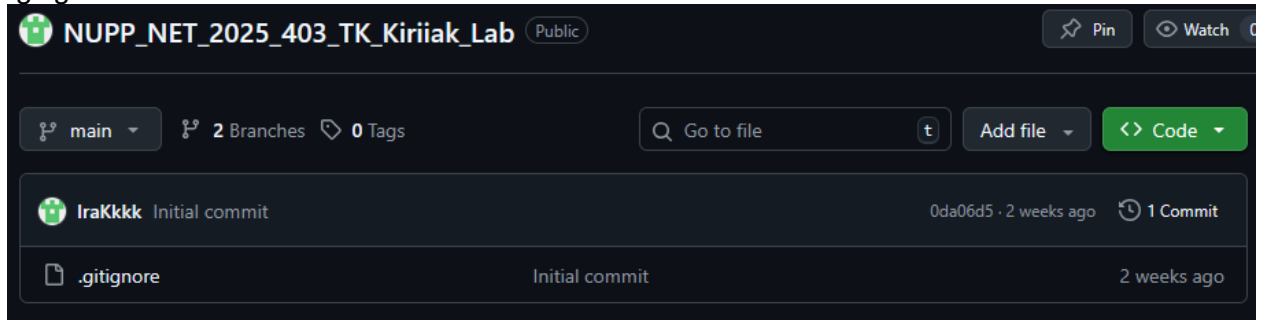
Богданович

Петро

Тема: Основи програмування у мові C#. ООП у мові C#. Базові конструкції та типи.

Передумова

Перед виконанням завдання необхідно створити публічний гіт репозиторій на сервісі [GitHub](#) із назвою NUPP_NET_2025_{Номер Групи}_TK_{Прізвище}_Lab і де буде розміщений вихідний код усіх готових лабораторних робіт. При створенні додати файл .gitignore за шаблоном Visual Studio:



Скопіюйте репозиторій на власну машину та створіть нову гілку **lab1**, в якому буде виконуватись дана лабораторна робота. Для здачі лабораторної роботи, необхідно буде створити пул реквест із гілки **lab1** у **master**.

Завдання

1. Створіть **свою ВЛАСНУ** модель(набір класів) на довільну тематику із 4 або більше класів, серед яких хоча б двоє класів наслідуються від якихось інших. Результуючі класи повинні знаходитись у проєкті {Назва тематики}.Common. Проєкти створювати всередині рішення(.sln) {Назва тематики}.

Приклад:

Кожен клас має мати від 3 властивостей:

```
namespace Games.Common;

// незалежний клас
public class Player
{
    public Guid Id { get; set; }
    public string Nickname { get; set; }

    // подія
    public delegate void GamePlayedHandler(string message);
    public event GamePlayedHandler? OnGamePlayed;

    // конструктор
    public Player(string nickname)
    {
        Id = Guid.NewGuid();
        Nickname = nickname;
    }
}
```

```

public void Play(Game game)
{
    OnGamePlayed?.Invoke($"{Nickname} почав грати у {game.Title}");
}
}

```

2. Додати у побудовану модель наступні базові конструкції:

- Методи;
- Конструктори;
- Статичні поля і конструктори;
- Делегати та Події;
- Статичний методи;
- Метод розширення.

Застосувати конструкцію 1 чи більше разів, над застосованою конструкцією залишити коментар:

```

class Program
{
    static void Main()
    {
        var crud = new CrudService<Game>();

        var g1 = new OnlineGame("Valorant", "Shooter", 2020, 10);
        var g2 = new OfflineGame("The Witcher 3", "RPG", 2015, true);

        crud.Create(g1);
        crud.Create(g2);

        var player = new Player("Ira");
        player.OnGamePlayed += msg => Console.WriteLine(msg);

        // Використання події та методу розширення
        player.Greet();
        player.Play(g1);

        Console.WriteLine("\n🎮 Усі ігри:");
        foreach (var game in crud.ReadAll())
            game.ShowInfo();

        // демонстрація збереження
        crud.Save("games.json");
        Console.WriteLine("\n✅ Дані збережено у games.json");
    }
}

```

Реалізувати дженерік CRUD сервіс, який буде зберігати дані у одній із вбудованих колекцій .NET та реалізовуватиме наступний інтерфейс:

```

public class CrudService<T> : ICrudService<T> where T : class
{
    private readonly List<T> _data = new();

    public void Create(T element) => _data.Add(element);

    public T? Read(Guid id)
    {
        var prop = typeof(T).GetProperty("Id");
        return _data.FirstOrDefault(x => prop?.GetValue(x)?.Equals(id) == true);
    }

    public IEnumerable<T> ReadAll() => _data;

    public void Update(T element)
    {
        Remove(element);
        Create(element);
    }
}

```

4. Створити новий проєкт {Назва тематики}.Console, із консольним застосунком, яким показуватиме як функціонує CRUD сервіс, наприклад створити новий об'єкт CRUD service, додати в нього нові об'єкти, вивести які об'єкти були додані і так далі.

```

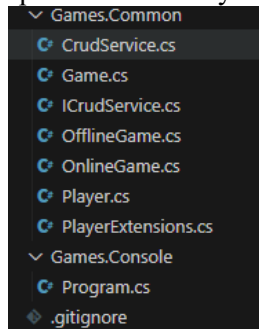
NUPP_NET_2025_403_TK_Kiriiak_Lab / Games.Console / Program.cs
IraKkkk Lab1: додано моделі Game, Player, CRUD сервіс

Code Blame 30 lines (23 loc) · 843 Bytes

1  using Games.Common;
2
3  class Program
4  {
5      static void Main()
6      {
7          var crud = new CrudService<Game>();
8
9          var g1 = new OnlineGame("Valorant", "Shooter", 2020, 10);
10         var g2 = new OfflineGame("The Witcher 3", "RPG", 2015, true);
11
12         crud.Create(g1);
13         crud.Create(g2);
14
15         var player = new Player("Ira");
16         player.OnGamePlayed += msg => Console.WriteLine(msg);
17
18         // Використання події та методу розширення
19         player.Greet();
20         player.Play(g1);
21
22         Console.WriteLine("\n 🎮 Усі ігри:");
23         foreach (var game in crud.ReadAll())
24             game.ShowInfo();
25
26         // демонстрація збереження
27         crud.Save("games.json");
28         Console.WriteLine("\n ✅ Дані збережено у games.json");
29     }
30 }

```

Готова лабораторна робота має наступну файлову ієрархію:

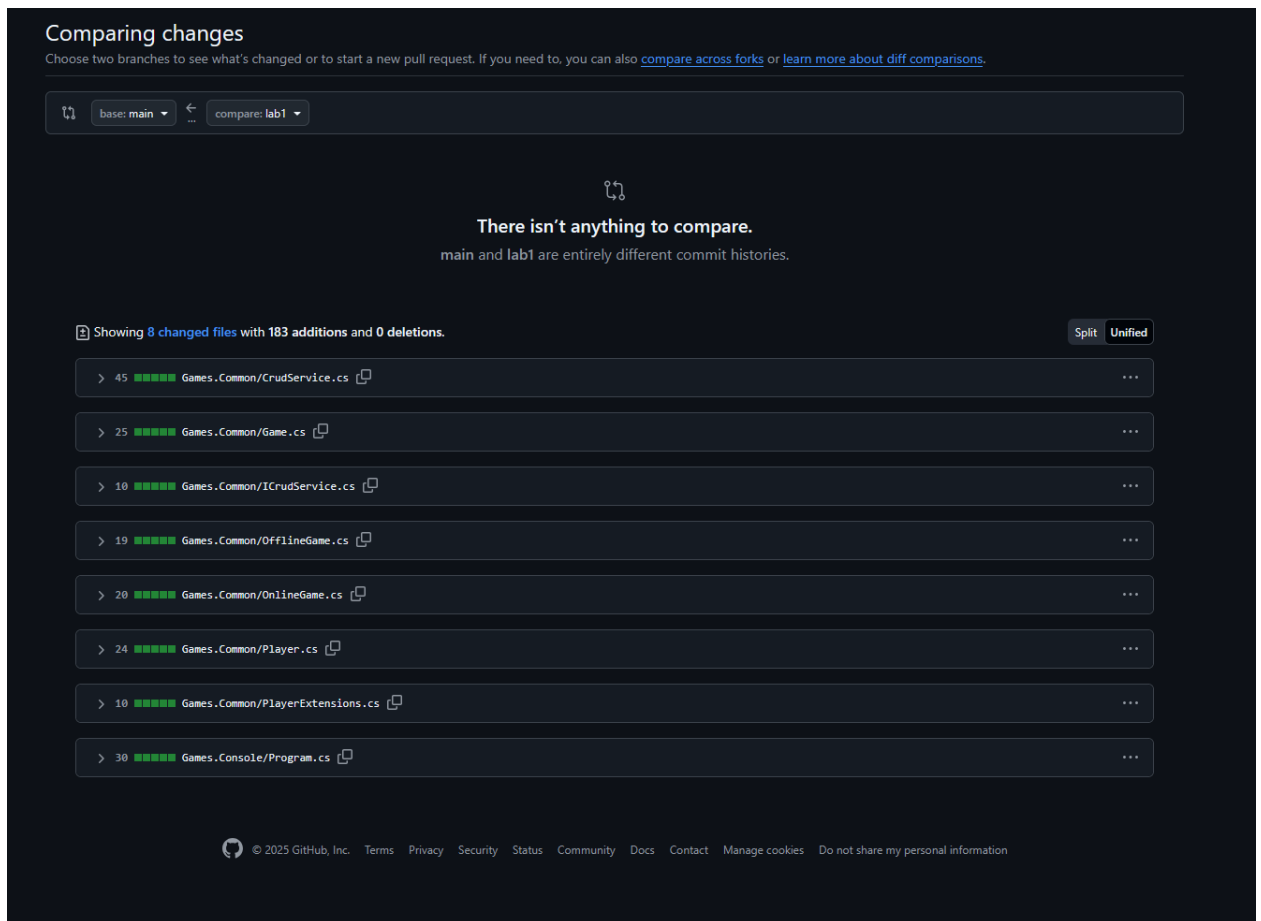


Додаткове завдання

Додати до CRUD сервісу методи Load(string FilePath) та Save(string FilePath), які будуть зберігати дані із сервісу у серіалізованому вигляді у файлі за шляхом FilePath та завантажувати дані із файлу.

```
public void Save(string filePath)
{
    var json = JsonSerializer.Serialize(_data);
    File.WriteAllText(filePath, json);
}

public void Load(string filePath)
{
    if (!File.Exists(filePath)) return;
    var json = File.ReadAllText(filePath);
    var loaded = JsonSerializer.Deserialize<List<T>>(json);
    if (loaded != null)
    {
        _data.Clear();
        _data.AddRange(loaded);
    }
}
}
```



Контрольні запитання

1. **Класи** — це шаблони для створення об'єктів, що містять дані і методи. **Структури** — легші типи даних, які зберігають прості значення. Класи — *reference type*, структури — *value type*. **Value типи** копіюють дані, а **reference типи** передають посилання на об'єкт.
2. **Абстрактний клас** — це клас, від якого не можна створити об'єкт, і який містить нереалізовані методи. Від звичайного класу відрізняється тим, що задає лише загальну поведінку для нащадків.
3. **Інтерфейс** — це набір методів без реалізації. Він визначає, що клас повинен робити, але не як. Від абстрактного класу відрізняється тим, що не містить реалізації та не має полів.
4. **Наслідування** — це механізм, коли один клас отримує властивості й методи іншого. **Модифікатори доступу**: public, private, protected, internal, protected internal, private protected.
5. **Статичні поля та методи** належать усьому класу, а не окремому об'єкту. Звичайні поля та методи належать конкретним екземплярам класу.
6. **Делегати** — це типи, які зберігають посилання на методи і дозволяють викликати їх через події або динамічно. Існують звичайні, анонімні, лямбда та вбудовані (Action, Func, Predicate) делегати.